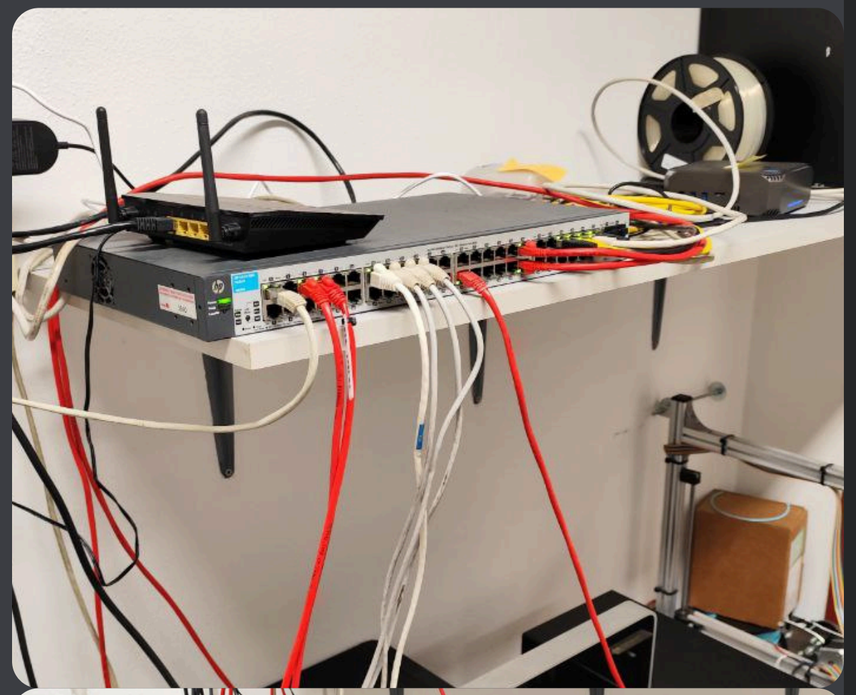
Cos’è >ADM staff?

ADMStaff è un gruppo di studenti che applicano ciò che imparano per sviluppare e mantenere servizi utili per la comunità universitaria. Non è un corso, ma un laboratorio dove si impara facendo.

L’università ci fornisce:

- Computer e hardware “enterprise”
- IP pubblici su cui hostare servizi
- Un laboratorio dove sperimentare

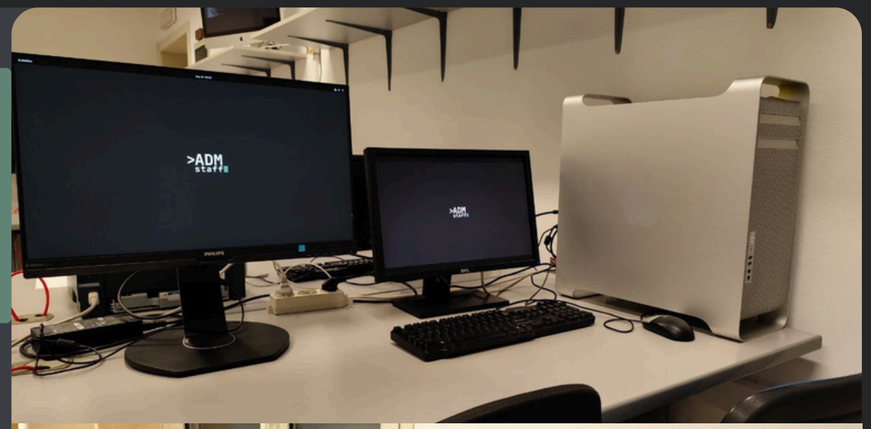


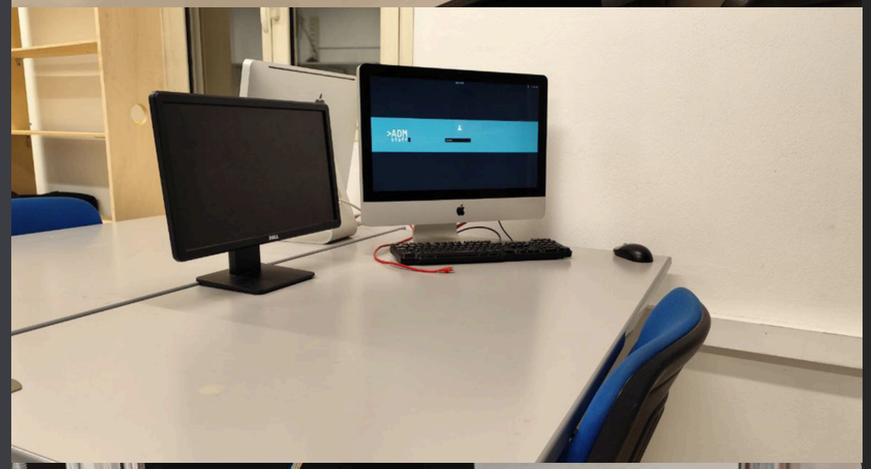
Il nostro laboratorio

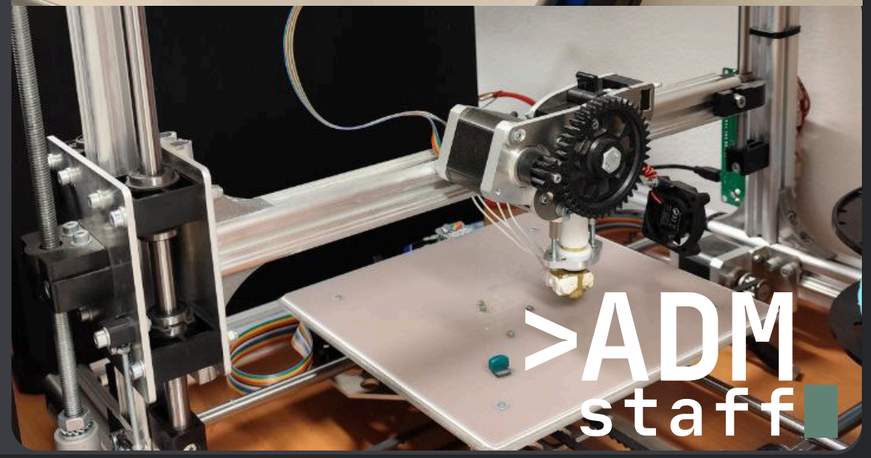
Ci troviamo in via Mura Anteo Zamboni 7, nel piano seminterrato, Aula 02.

Perché venire in laboratorio?

- Spazi dedicati allo studio e condivisione
- Postazioni con **ADMWiFi** di libero accesso
- Connessione ethernet senza limitazioni di porte
- CoffeeStaff: macchinetta del caffè IoT automatizzata ESPresso32







Cosa facciamo oggi?

Proveremo a farvi sentire parte di ADM, mettendo in pratica quello che facciamo in laboratorio.

>ADM staff

Che cos’è l’informatica?

>ADM staff

Cosa non è l’informatica?

- Montare e smontare computer
- Installare pacchetti software
- Semplicemente navigare in internet
- Conoscere tanti linguaggi di programmazione
- "Programmare e basta"

>ADM staff

Dividiamoci in gruppi

>ADM staff

Dividiamoci in gruppi

0. prendete un case
1. prendete un disco
2. prendete una RAM
3. prendete le periferiche
4. prendete un kit cavi
5. prendete gli strumentopoli che vi servono

Montiamo ora una macchina
per mettere insieme le mani in pasta nell’informatica.

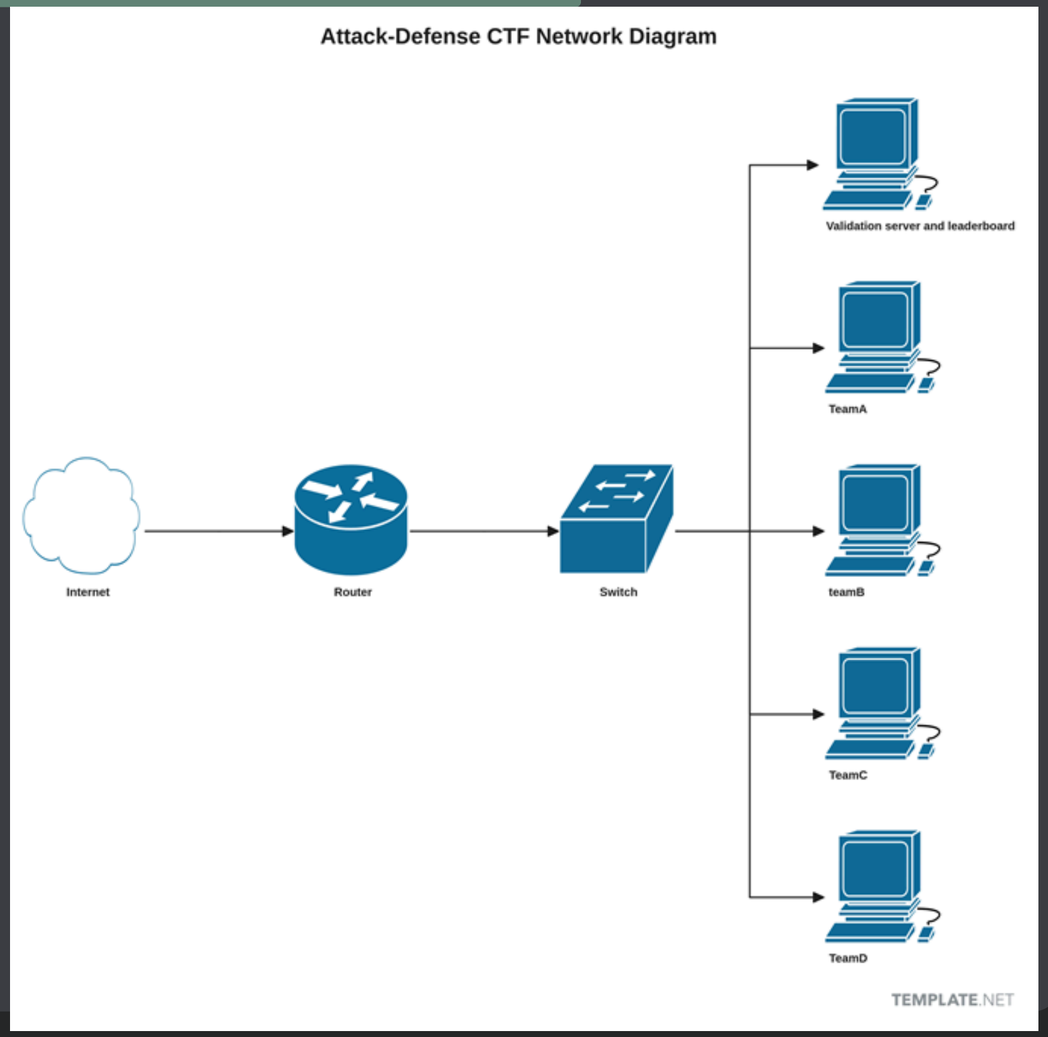
>ADM staff

github.com/fabiomurer/UGU-2026

>ADM staff

Obiettivo

- Collegarsi al server di gioco dei computer avversari
- Risolvere le sfide
- Quando una sfida viene risolta produce un codice (Flag) che ne testimonia il completamento
- Inviare la Flag al server di validazione per incrementare il punteggio



exploit_client.py

server delle sfide

- Aspetta che un utente si connetta al server di validazione.
- Questo farà diverse domande, se la risposta è giusta invia la flag al server di validazione e passa a quella successiva.
- Se la risposta è sbagliata chiude la connessione.

```python
async def handle_client(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    async def timeout():
        await asyncio.sleep(10)
        try:
            writer.close()
            await writer.wait_closed()
        except Exception:
            pass

    timeout_task = asyncio.create_task(timeout())
    try:
        await send_line(writer, f"Welcome to the quiz! Team: {TEAM_ID}")

        # Step 1
        await step1(reader, writer)
        await send_flag(writer, TEAM_ID, 1)

        # Step 2
        await step2(reader, writer)
        await send_flag(writer, TEAM_ID, 2)

        # Step 3
        await step3(reader, writer)
        await send_flag(writer, TEAM_ID, 3)

        await send_line(writer, "Congratulations! You've completed the quiz.")
        return
    finally:
        timeout_task.cancel()
        try:
            writer.close()
            await writer.wait_closed()
        except Exception:
            pass
```

exploit_client.py

script che risolve le sfide

- Si connette al server del team, legge la sfida e la risolve
- Se la risposta è giusta, da la flag in output
- Il codice che avete è incompleto… dovrete scrivere il codice mancante per risolvere le sfide.

```python
def solve_quiz(host: str, port: int, validator_url: str, my_team: str, victim: str):
    s = socket.create_connection((host, port), timeout=5)

    # banner
    recvline(s)

    # Step 1
    if not solve_step1(s):
        raise Exception("Step 1 failed")
    flag1 = get_flag(s)
    print(f"[+] Got flag 1: {flag1}")
    if validator_url:
        submit_flag(validator_url, my_team, victim, 1, flag1)

    # Step 2
    if not solve_step2(s):
        raise Exception("Step 2 failed")
    flag2 = get_flag(s)
    print(f"[+] Got flag 2: {flag2}")
    if validator_url:
        submit_flag(validator_url, my_team, victim, 2, flag2)

    # Step 3
    if not solve_step3(s):
        raise Exception("Step 3 failed")
    flag3 = get_flag(s)
    print(f"[+] Got flag 3: {flag3}")
    if validator_url:
        submit_flag(validator_url, my_team, victim, 3, flag3)
```

exploit_client.py

completare le sfide

STEP 1

- Completare le funzioni solve_step(n):
- Vediamo la prima domanda:
- Utilizziamo il linguaggio Python
- Per capire come utilizzarlo:
  - https://learnxinyminutes.com/python/
  - https://automatetheboringstuff.com/

```python
async def handle_client(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    async def timeout():
        await asyncio.sleep(10)
        try:
            writer.close()
            await writer.wait_closed()
        except Exception:
            pass

    timeout_task = asyncio.create_task(timeout())
    try:
        await send_line(writer, f"Welcome to the quiz! Team: {TEAM_ID}")

        # Step 1
        await step1(reader, writer)
        await send_flag(writer, TEAM_ID, 1)

        # Step 2
        await step2(reader, writer)
        await send_flag(writer, TEAM_ID, 2)

        # Step 3
        await step3(reader, writer)
        await send_flag(writer, TEAM_ID, 3)

        await send_line(writer, "Congratulations! You've completed the quiz.")
        return
    finally:
        timeout_task.cancel()
        try:
            writer.close()
            await writer.wait_closed()
        except Exception:
            pass
```

Consegnare le flag

- La flag viene inviata automaticamente mandando i dati al server di validazione, questo può essere fatto anche manualmente con il comando curl
- Il server vi risponderà se la flag è corretta o no

```
curl -s -X POST http://<ip>:8000/submit \
  -H 'Content-Type: application/json' \
  -d '{"team":"<myTeam>", "victim":"<attackedTeam>", "step":1, "flag":"<flag>"}'
```

>ADM staff

exploit_client.py

completare le sfide

STEP 2

- Indovina il numero random
- Ci sono un numero finito di prove
- Approccio?
  - Prova numeri random
    → c'è di meglio!
  - Ricerca binaria :)

```python
async def step2(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    x = random.randint(0, MAXNUM)
    await send_line(writer, f"guess the number? [ 0, {MAXNUM} ]")

    # loop di tentativi
    for _ in range(int(math.log(MAXNUM, 2) + 1)):
        guessed_x = int(((await read_line(reader)).strip()))

        if guessed_x < x:
            await send_line(writer, "higher")
        elif guessed_x > x:
            await send_line(writer, "lower")
        else:
            await check(writer, True)
            return

    await send_line(writer, "too many attempts")
    await check(writer, False)
```

exploit_client.py

completare le sfide

STEP 3

- Si può chiedere a un oracolo di cifrare un messaggio.
- Per risolvere la sfida bisogna trovare la chiave segreta che usa l'oracolo per cifrare.
- La cifratura viene applicata con la funzione XOR https://en.wikipedia.org/wiki/Exclusive_or
- L'XOR ha delle proprietà interessanti… utilizzale a tuo vantaggio!

```python
async def step3(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    # Genera un secret casuale di 8 byte
    secret = os.urandom(8)
    await send_line(
        writer, "XOR oracle: send 'enc <hex_msg>' or 'secret <hex_secret>' (8 bytes)"
    )

    while True:
        line = (await read_line(reader)).strip()
        if line.startswith("enc "):
            msg_hex = line[4:]
            try:
                msg = bytes.fromhex(msg_hex)

                # Previene l'exploit con zeri: rifiuta il messaggio se contiene byte nulli
                if b"\x00" in msg:
                    await send_line(writer, "no zeros allowed!")
                    continue

                # Esegue lo XOR tra il messaggio e il secret
                res = bytes([msg[i] ^ secret[i % len(secret)] for i in range(len(msg))])
                await send_line(writer, res.hex())
            except ValueError:
                await send_line(writer, "invalid hex")

        elif line.startswith("secret "):
            guess_hex = line[7:]
            if guess_hex == secret.hex():
                await check(writer, True)
                return
            else:
                await check(writer, False)
                return
        else:
            await send_line(writer, "unknown command")
```

exploit_client.py

completare le sfide

STEP 3

Qualche tips sullo XOR:

- significa OR Esclusivo: o uno o l’altro, ma non entrambi.
- XOR ⊕ è un’operazione che confronta due valori 0 e 1:
  - Restituisce 1 (vero) solo se i due valori sono diversi
  - Restituisce 0 (falso) se sono uguali
- è **invertibile**:
  - Se fai: A ⊕ B ⊕ B = A

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

exploit_client.py

completare le sfide

STEP 4

- Una stringa random viene trasformata in un programma https://esolangs.org/wiki/Brainfuck
- Per risolvere la sfida bisogna creare un interprete per questo linguaggio, eseguendo il programma in output si avrà la stringa.

```
async def step4(reader: asyncio.StreamReader,
                writer: asyncio.StreamWriter):
    secret = randomword(8)
    bfsecret = text_to_brainfuck(secret)

    await send_line(
        writer, "Can you tell me what this brainfuck program outputs?"
    )
    await send_line(writer, bfsecret)

    guessed = (await read_line(reader)).strip()

    await check(writer, secret == guessed)
```

Hello world in Brainfuck:

```
++++++++[>++++[>++>+++>+++>+<<<<-]>+>+>->>+[<]<-]>>.>---.+++++++..+++.>>.<-.<.++
+.------.--------.>>+.>++.
```

>ADM staff

t.me/admstaff_Chat (laboratori ADM)

t.me/Zwirgio (Fabio Murer)

t.me/G0llum05 (Mattia Graziani)

t.me/alii_benatti (Alice Benatti)